		conversions.	educate teams on common pitfalls.
Error handling	Use `try`/`catch`, explain sync vs async errors.	Handle errors in Promises/async code; create custom error types.	Architect global error handling; implement error boundaries; add monitoring systems.

In the table, each topic is covered by three levels:

- *Fundamentals*—Covers basic definitions and core concepts. Candidates should understand key terms and explain straightforward behaviors, forming the foundation for further learning.
- *Applied skills*—Focuses on applying fundamentals to real-world coding challenges. Candidates demonstrate debugging, handling edge cases, and effective use of language features in practical scenarios.
- *Advanced*—Involves deep mastery and architectural insight. Candidates tackle complex bugs, optimize performance, design scalable APIs, and build maintainable systems—skills expected from senior developers.

With the foundation of execution and scope in mind, let’s dive into some common interview questions to see how these concepts are applied in practice.

Initial question (fundamental skill):"What is the output of the variable `message` as shown in Listing 3.1?"

Listing 3.1 Hoisting—an example of execution context and scope

```
'use strict';
function greet() {
  console.log(message);
  let message = "Hello, world!";
}
greet();
```

Analysis: This question tests whether a candidate can clearly explain the